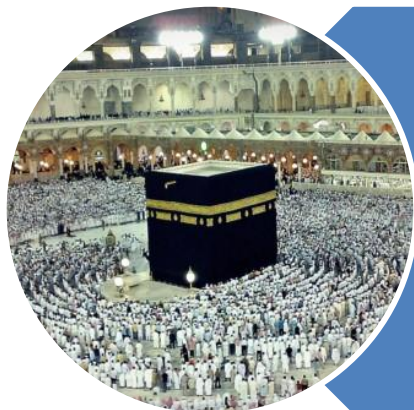


رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Recursion

With “Numeric” values

Sum of “n” Integers

Example 03

C

P

C

S

2

0

4

4

Sum of Integers – Generic Formula

- $\text{Sum}(5) = 5+4+3+2+1$
 $= 5+ (4+3+2+1)$
 $= 5 + \text{Sum}(4)$
- $\text{Sum}(4) = 4+3+2+1$
 $= 4+(3+2+1)$
 $= 4 + \text{Sum}(3)$
- $\text{Sum}(3) = 3+2+1$
 $= 3+ (2+1)$
 $= 3 + \text{Sum}(2)$
- Thus we observed
 - $\text{Sum}(5) = 5 + \text{Sum}(4)$
 - $\text{Sum}(4) = 4 + \text{Sum}(3)$
 - $\text{Sum}(3) = 3 + \text{Sum}(2)$
- So, for any value “n”
 $\text{Sum}(n) = n + \text{Sum}(n - 1)$

C

P

C

S

2

0

4

5

Sum of Integers – **Base Case**

- For non-negative integers
 - If we add first “0” integers then the sum will be 0 (zero)
 - $\text{Sum}(0) = 0$

OR

- If we add first “1” integers then the sum will be 1 (One)
- $\text{Sum}(1) = 1$

C

P

C

S

2

0

4

6

Sum of Integers – Code

- Generic Formula

$\text{Sum}(n) = n + \text{Sum}(n - 1)$

- Base Case

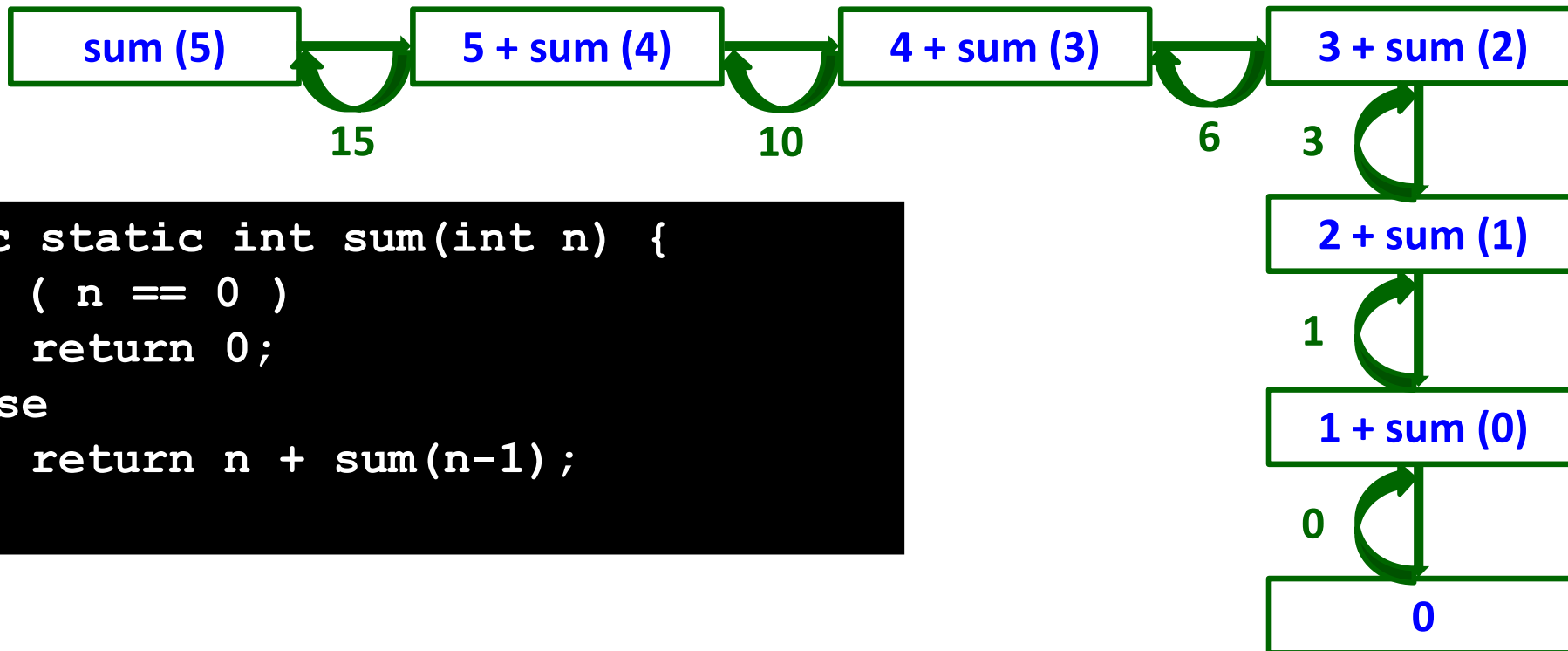
$\text{Sum}(0) = 0$

```
public static int sum(int n) {  
    if ( n == 0 )  
        return 0;  
    else  
        return n + sum(n-1);  
}
```

- Initial Method Call

$\text{sum}(n);$

Sum of Integers – Code Tracing sum(5)



```
public static int sum(int n) {
    if ( n == 0 )
        return 0;
    else
        return n + sum(n-1);
}
```

Result = 15

Power function i.e. b^e

Example 04

C

P

C

S

2

0

4

Power Function - Introduction

- Base “**b**”
- Exponent “**e**”
 - Exponent is always non-negative
- Calculate “**b^e**”
- Represent b^e as **power(b, e)**

C

P

C

S

2

0

4

Power Function – Generic Formula

- Power(2,5)

- $= 2 * 2 * 2 * 2 * 2$
- $= 2 * (2 * 2 * 2 * 2)$
- $= 2 * \text{Power}(2,4)$

- Power(2,4)

- $= 2 * 2 * 2 * 2$
- $= 2 * (2 * 2 * 2)$
- $= 2 * \text{Power}(2,3)$

- Thus we observed

- $\text{Power}(2,5) = 2 * \text{Power}(2,4)$
- $\text{Power}(2,4) = 2 * \text{Power}(2,3)$
- $\text{Power}(2,3) = 2 * \text{Power}(2,2)$

- So, for any value “b” and “e”

- **$\text{Power}(b,e) = b * \text{Power}(b,e-1)$**

C

P

C

S

2

0

4

Power Function— **Base Case**

- For any base with exponent “0”, the result will be 1 (one)

- Exponent = 0, return 1

OR

- For any base with exponent “1”, the result will be the base i.e b

- Exponent = 1, return base

C

P

C

S

2

0

4

Power Function— Code

- Generic Formula

$\text{Power}(b, e) = b * \text{Power}(b, e - 1)$

- Base Case

$\text{Power}(b, 0) = 1$

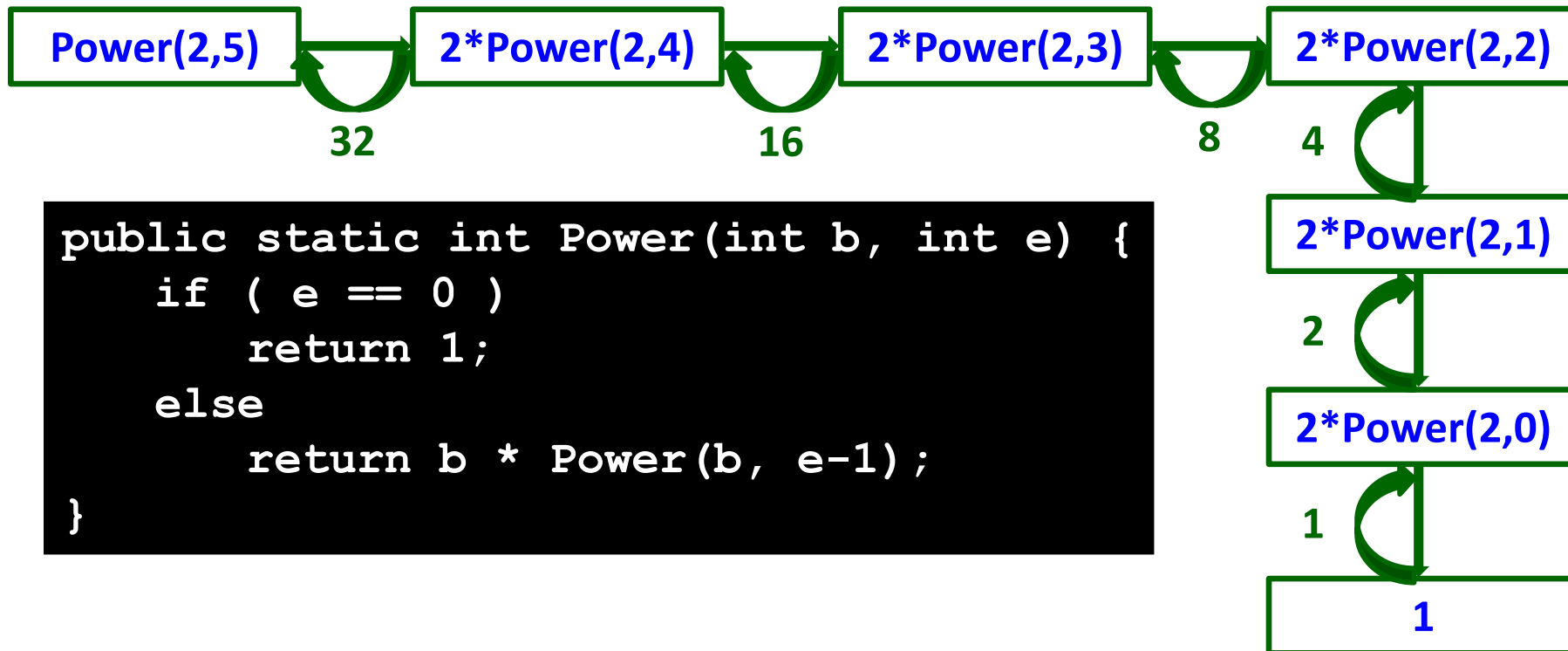
```
public static int Power(int b, int e) {  
    if ( e == 0 )  
        return 1;  
    else  
        return b * Power(b, e-1);  
}
```

- Initial Method Call

$\text{Power}(b, e);$

Power Function– Code Tracing

Power(2, 5)



```
public static int Power(int b, int e) {
    if ( e == 0 )
        return 1;
    else
        return b * Power(b, e-1);
}
```

Result = 32

Power function i.e. b^e

Fast Exponentiation

Example 04

C

P

C

S

2

0

4

Fast Exponentiation- Introduction

- Base “**b**”
- Exponent “**e**”
 - Exponent is always non-negative
- Calculate “**b^e**”
 - $b^e = b * b^{e-1}$ (Slow)
 - # of multiplications required (**$e-1$**)
 - $b^e = b^{e/2} * b^{e/2}$ (Fast)
 - # of multiplications required (**$\log_2 e$**)

C

P

C

S

2

0

4

Fast Exponentiation - **Introduction**

- Remembering the laws of exponents
 - $2^8 = 2^4 * 2^4$
 - we can calculate 2^8 with one multiplication
- What is 2^4 ?
 - $2^4 = 2^2 * 2^2$
 - $2^2 = 2^1 * 2^1$
- So we've calculated 2^8 using only **three multiplications**
 - Previously it was $2 * 2 * 2 * 2 * 2 * 2 * 2 * 2$
 - Needs **7 multiplications**

Fast Exponentiation – **Generic Formula**

- 2^8
○ $= 2^4 * 2^4$
- 2^4
○ $= 2^2 * 2^2$
- 2^2
○ $= 2^1 * 2^1$
- $2^9 = 2^8 * 2^1$
○ $= 2^4 * 2^4 * 2^1$

- So, for any value base “a” and exponent “n”

$$○ a^n = a^{n/2} * a^{n/2}$$

$$a^n = \begin{cases} a^{n/2} (a^{n/2}) & \text{if n is even} \\ a^{n/2} (a^{n/2}) (a) & \text{if n is odd} \end{cases}$$

C

P

C

S

2

0

4

Fast Exponentiation – **Base Case**

- For any base with exponent “0”, the result will be 1 (one)

- Exponent = 0, return 1

OR

- For any base with exponent “1”, the result will be the base

- Exponent = 1, return base

Power Function– Code

- Initial Call

powerF(b, e);

- Base Case

powerF(b, 0) = 1

powerF(b, 1) = b

- Generic Formula

powerF(b, e) = powerF (b*b, e/2) (even)

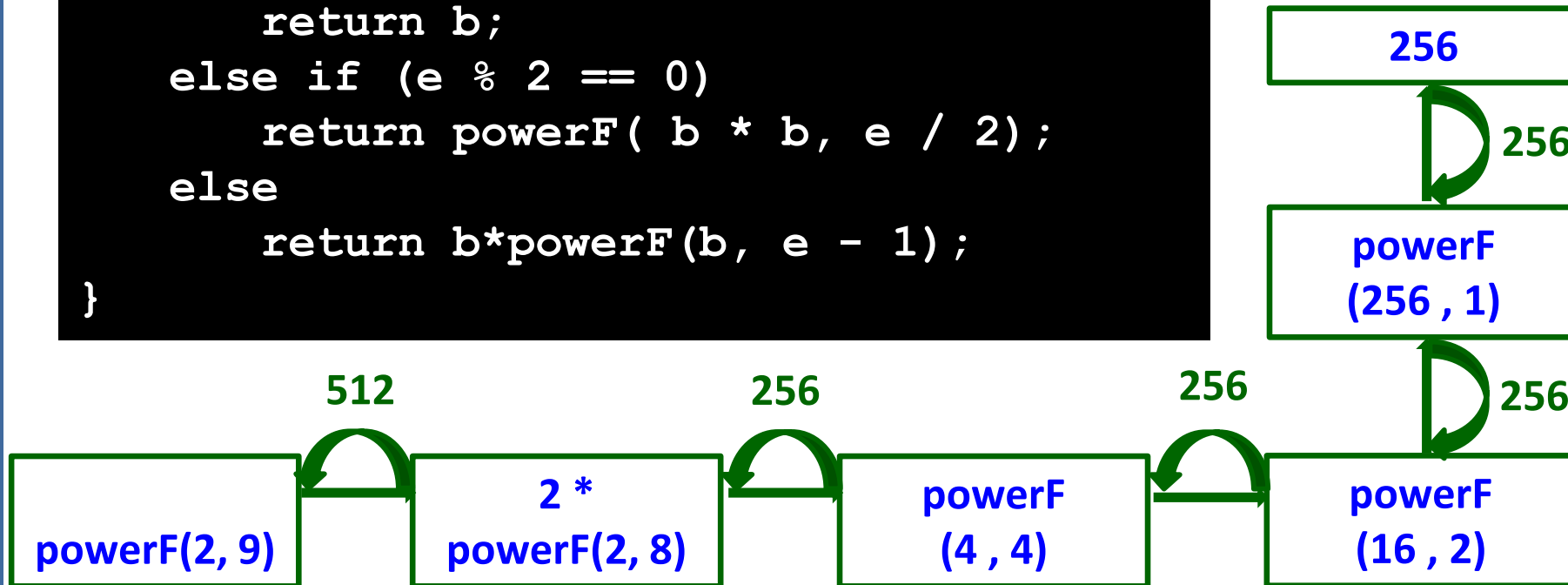
powerF(b, e) = b * Power (b, e – 1) (odd)

```
public static int powerF(int b, int e) {
    if (e == 0)
        return 1;
    else if (e == 1)
        return b;
    else if (e % 2 == 0)
        return powerF( b * b, e / 2 );
    else
        return b*powerF(b, e - 1);
}
```

Power Function– Code Tracing powerF(2, 9)

```
public static int powerF(int b, int e) {
    if (e == 0)
        return 1;
    else if (e == 1)
        return b;
    else if (e % 2 == 0)
        return powerF( b * b, e / 2);
    else
        return b*powerF(b, e - 1);
}
```

Result = 512



Multiplying TWO numbers

(Using addition)

Example 05

C

P

C

S

2

0

4

Multiplication - Introduction

- Both numbers are *“non-negative”*
- First number *“first”*
- Second number *“second”*
- Calculate **“first * second”** i.e. $6 * 5$
 - $6 * 5 = 6 + 6 + 6 + 6 + 6$
- Represent $6 * 5$ as **multi (first, second)**

C

P

C

S

2

0

4

Multiplication – Generic Formula

- Multi(6,5)

- = $6+6+6+6+6$
- = $6+(6+6+6+6)$
- = $6+Multi(6,4)$

- Multi(6,4)

- = $6+6+6+6$
- = $6+(6+6+6)$
- = $6+Multi(6,3)$

- Thus we observed

- $Multi(6,5) = 6 + Multi(6,4)$
- $Multi(6,4) = 6 + Multi(6,3)$
- $Multi(6,3) = 6 + Multi(6,2)$

- So, for any value
“first” and “second”

- **$Multi(f, s) = f + Multi(f, s-1)$**

C

P

C

S

2

0

4

Multiplication – **Base Case**

- If any of the value, either “first” number or “second” number is “0”, then the result will be 0 (zero)
 - multi (first, 0), return 0
- OR**
- multi (0, second), return 0

Multiplication – Code

- Generic Formula

Multi (first, second)

= first + Multi (first, second – 1)

- Base Case

Multi(first, 0) = 0 or

Multi(0, second) = 0

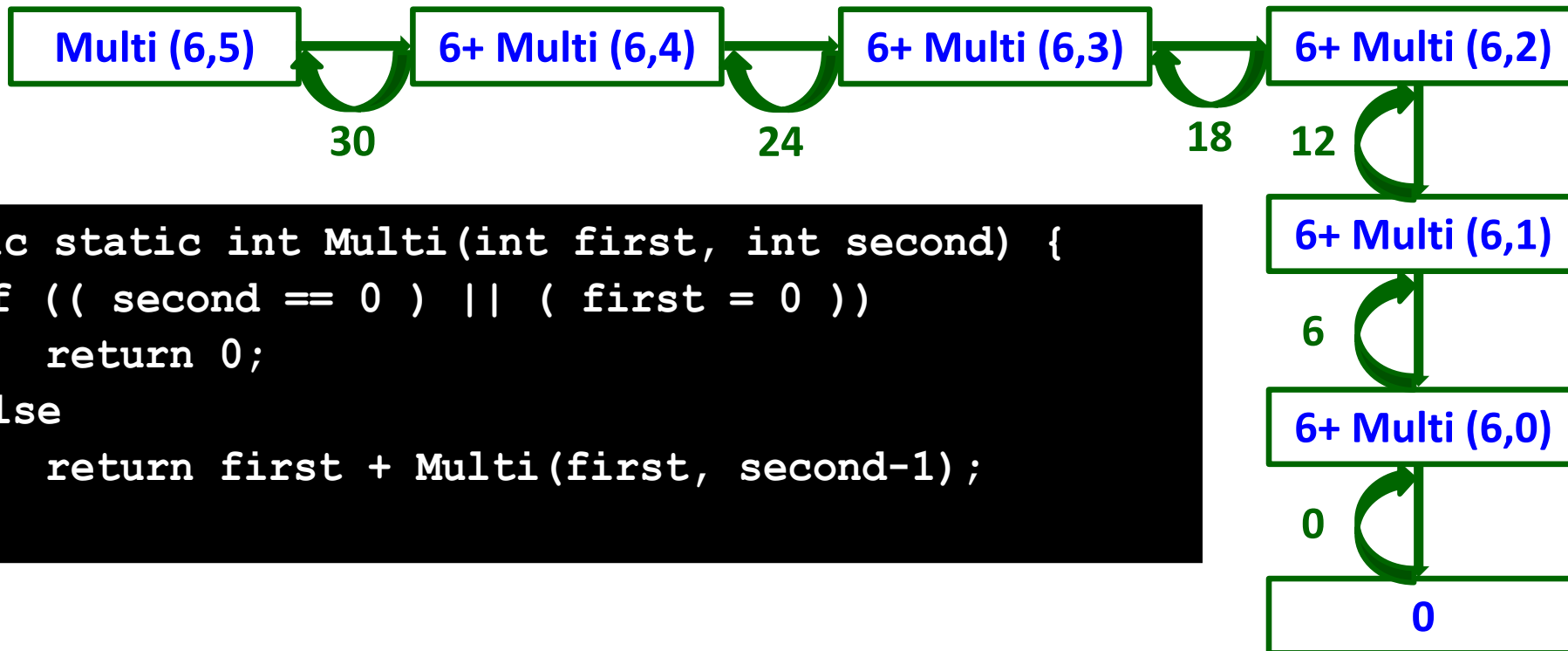
- Initial Method Call

Multi(first, second);

```
public static int Multi(int first, int second) {
    if (( second == 0 ) || ( first = 0 ))
        return 0;
    else
        return first + Multi(first, second-1);
}
```

Multiplication – Code Tracing

Multi (6, 5)



```
public static int Multi(int first, int second) {
    if (( second == 0 ) || ( first = 0 ))
        return 0;
    else
        return first + Multi(first, second-1);
}
```

Result = 30

Fibonacci Sequence

Example 06

C

P

C

S

2

0

4

Fibonacci - Introduction

- First term is “1”
- Second term is “1”
- Every next term is **sum of previous two terms**
- Fibonacci Sequence
 - 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89,
- “**nth**” term of Fibonacci sequence is represented as **Fib(n)**

C

P

C

S

2

0

4

Fibonacci – Generic Formula

- Fib(6)
 - = Fib(5) + Fib(4)
- Fib(5)
 - = Fib(4) + Fib(3)
- Fib(4)
 - = Fib(3) + Fib(2)
- Fib(3)
 - = Fib(2) + Fib(1)
- Thus we observed
 - Fib(6) = Fib(5) + Fib(4)
 - Fib(5) = Fib(4) + Fib(3)
 - Fib(4) = Fib(3) + Fib(2)
- So, for any value "n"
Fib(n) = Fib(n-1) + Fib(n-2)

C

P

C

S

2

0

4

Fibonacci – Base Case

- First two terms of the Fibonacci sequence are 1, so

- $\text{fib}(1) = 1$

OR

- $\text{fib}(2) = 1$

C

P

C

S

2

0

4

Fibonacci– Code

- Generic Formula

$$\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$$

- Base Case

$$\text{fib}(1) = 1$$

$$\text{fib}(2) = 1$$

- Initial Method Call

fib(n);

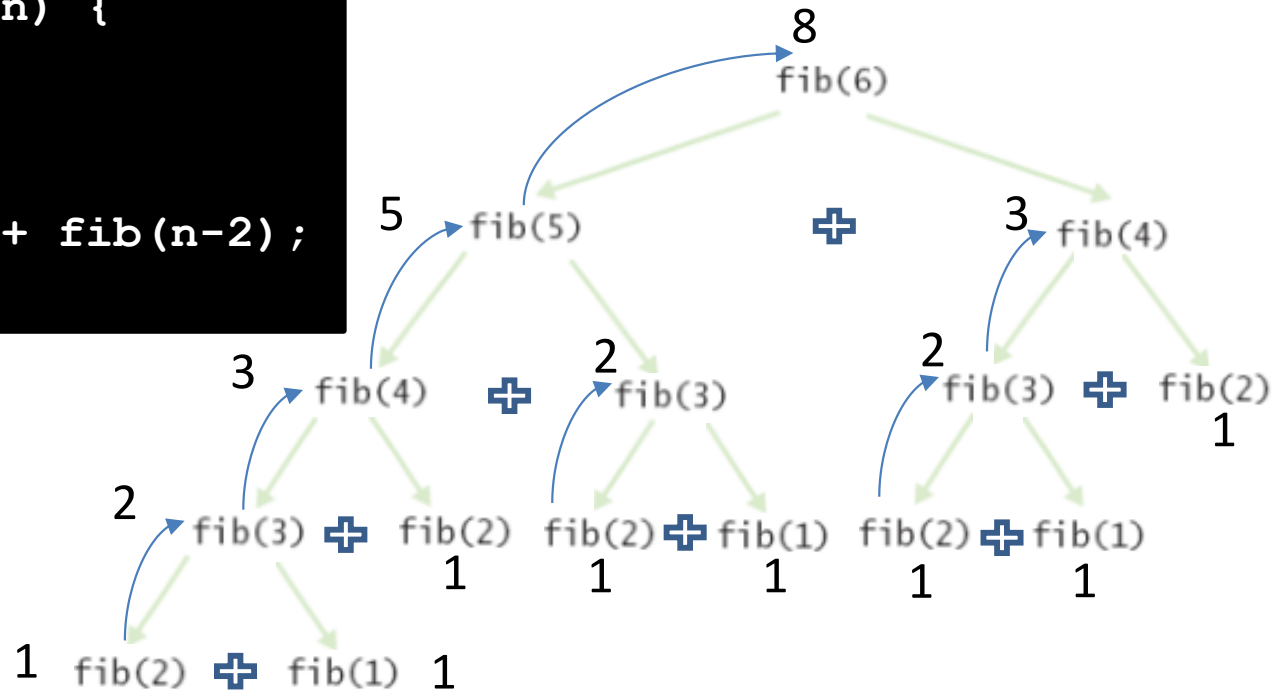
```
public static int fib(int n) {
    if (n <= 2)
        return 1;
    else
        return fib(n-1) + fib(n-2);
}
```

"No Bonus" marks for this course anymore

Fibonacci – Code Tracing

```
public static int fib(int n) {
    if (n <= 2)
        return 1;
    else
        return fib(n-1) + fib(n-2);
}
```

fib (6)



1, 1, 2, 3, 5, 8, 13, 21, 34,

Result = 8

Sum the digits of a number

Example 07

C

P

C

S

2

0

4

Sum of digits - Introduction

- The number (**n**) is “*non-negative*”
- Yield the rightmost digit
 - **$n \% 10$ = rightmost digit**
 - **$6357 \% 10 = 7$**
- Remove the rightmost digit
 - **$n / 10$ = number without rightmost digit**
 - **$6357 / 10 = 635$**
- Function represented as **sumDigits (n)**

Sum of digits – Generic Formula

- $\text{sum}(6357)$
 - $= 6+3+5+7$
 - $= (635) + 7$
 - $= \text{sum}(635)+7$
- $\text{sum}(635)$
 - $= 6+3+5$
 - $= (63) + 5$
 - $= \text{sum}(63) + 5$
- $\text{sum}(63)$
 - $= 6+3$
 - $= (6) + 3$
 - $= \text{sum}(6) + 3$
- Thus we observed
 - $\text{Sum}(6357) = \text{sum}(635) + 7$
 - $\text{Sum}(635) = \text{sum}(63) + 5$
 - $\text{Sum}(63) = \text{sum}(6) + 3$
- So, for any value “n”
- **$\text{sum}(n) = \text{sum}(n/10) + n\%10$**

C

P

C

S

2

0

4

Sum of digits – **Base Case**

- If the number is “0” then the sum of digits will be 0 (zero).

○ $n = 0$, return 0

C

P

C

S

2

0

4

Sum of digits – Code

- Initial Method Call

$\text{sumDigits}(n);$

- Base Case

$\text{sumDigits}(0) = 0$

```
public static int sumDigits(int n) {  
    if ( n == 0 )  
        return 0;  
    else  
        return sumDigits(n/10) + (n%10);  
}
```

- Generic Formula

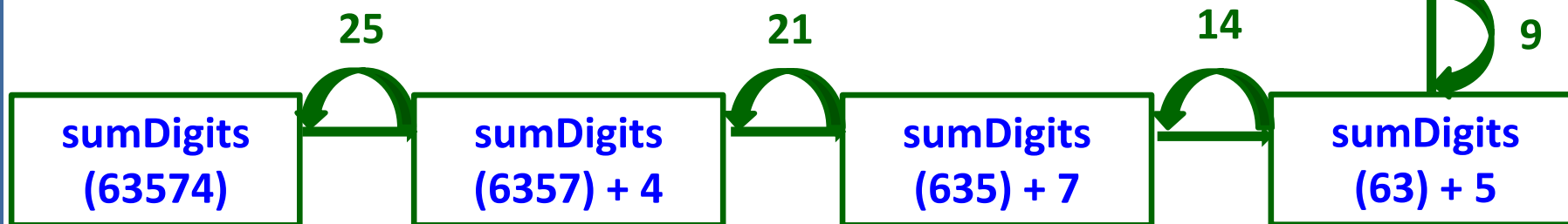
$\text{sumDigits}(n) = \text{sumDigits}(n/10) + (n\%10)$

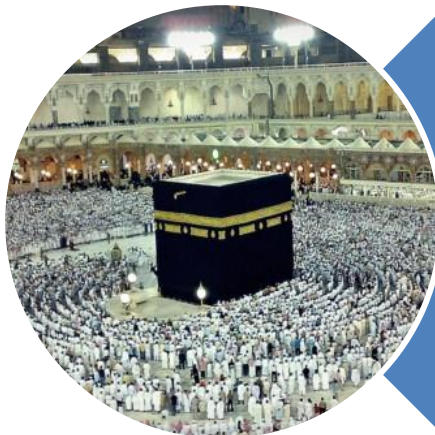
Sum of digits – Code Tracing

sumDigits (63574)

Result = 25

```
public static int sumDigits(int n) {
    if ( n == 0 )
        return 0;
    else
        return sumDigits(n/10) + (n%10);
}
```





Allah (Alone) is Sufficient for us,
and He is the Best Disposer of
affairs (for us). **(Quran 3 : 173)**